

Inference Time Scaling

COMP4551

Binhang Yuan

Inference Time Scaling Law

Infinite Monkey Theorem

“A monkey hitting keys independently and at random on a typewriter keyboard for an infinite amount of time will almost surely type any given text, including the complete works of William Shakespeare.”



[Image generated by Chat-GPT-4o-mini]

Large Language Monkeys:

Scaling Inference Compute with Repeated Sampling

Using repeated sampling for scaling inference compute to improve LLM performance (e.g., reasoning).

arXiv:2407.21787v3 [cs.LG] 30 Dec 2024

Large Language Monkeys: Scaling Inference Compute with Repeated Sampling

Bradley Brown^{*†‡}, Jordan Juravsky^{*†}, Ryan Ehrlich^{*†}, Ronald Clark[‡], Quoc V. Le[§],
Christopher Ré[†], and Azalia Mirhoseini^{†§}

[†]Department of Computer Science, Stanford University

[‡]University of Oxford

[§]Google DeepMind

bradley.brown@cs.ox.ac.uk, jbj@stanford.edu, ryanehrlich@cs.stanford.edu,
ronald.clark@cs.ox.ac.uk, qvl@google.com, chrismre@stanford.edu,
azalia@stanford.edu

Abstract

Scaling the amount of compute used to train language models has dramatically improved their capabilities. However, when it comes to inference, we often limit models to making only one attempt at a problem. Here, we explore inference compute as another axis for scaling, using the simple technique of repeatedly sampling candidate solutions from a model. Across multiple tasks and models, we observe that coverage – the fraction of problems that are solved by any generated sample – scales with the number of samples over four orders of magnitude. Interestingly, the relationship between coverage and the number of samples is often log-linear and can be modelled with an exponentiated power law, suggesting the existence of inference-time scaling laws. In domains like coding and formal proofs, where answers can be automatically verified, these increases in coverage directly translate into improved performance. When we apply repeated sampling to SWE-bench Lite, the fraction of issues solved with DeepSeek-Coder-V2-Instruct increases from 15.9% with one sample to 56% with 250 samples, outperforming the single-sample state-of-the-art of 43%. In domains without automatic verifiers, we find that common methods for picking from a sample collection (majority voting and reward models) plateau beyond several hundred samples and fail to fully scale with the sample budget.

1 Introduction

The ability of large language models (LLMs) to solve coding, mathematics, and other reasoning tasks has improved dramatically over the past several years [47, 11, 2, 4]. Scaling the amount of training compute through bigger models, longer pre-training runs, and larger datasets has been a consistent driver of these gains [27, 37, 28].

In contrast, a comparatively limited investment has been made in scaling the amount of computation used during inference. Larger models do require more inference compute than smaller ones, and prompting techniques like chain-of-thought [61] can increase answer quality at the cost of longer (and therefore more computationally expensive) outputs. However, when interacting with LLMs, users and developers often restrict models to making only one attempt when solving a problem.

^{*} Title inspired by https://en.m.wikipedia.org/wiki/Infinite_monkey_theorem.

^{Code:} https://github.com/ScalingIntelligence/large_language_monkeys.

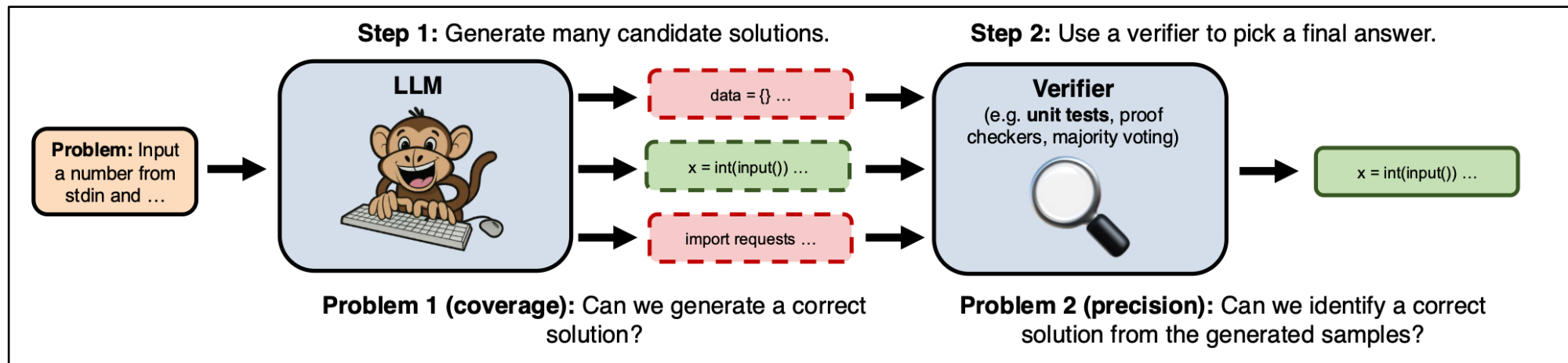
^{Data:} https://huggingface.co/datasets/ScalingIntelligence/monkey_business.

^{*} Equal Contribution. Work done by BB as a visiting researcher at Stanford.

<https://arxiv.org/pdf/2407.21787>

Scaling Repeated Sampling

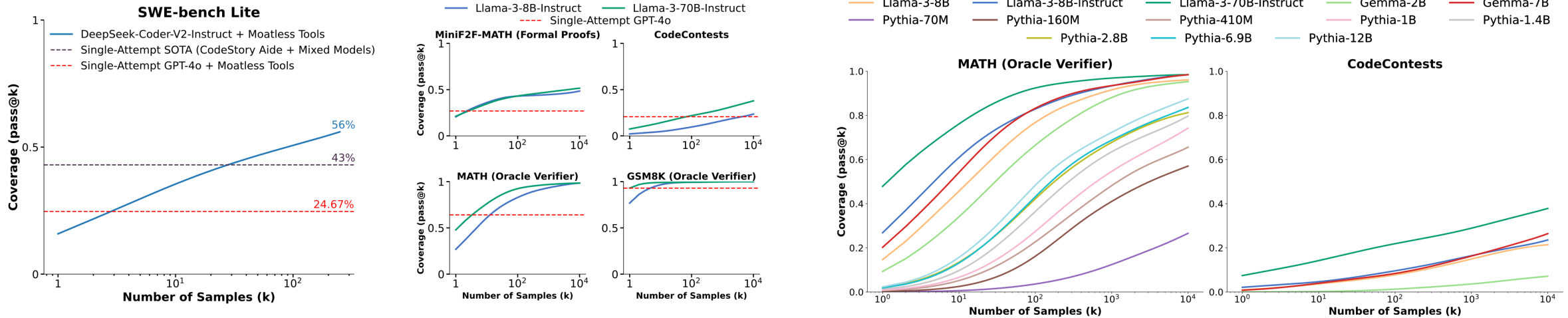
- In a standard setting: we usually measure the task performance by the success rate: the fraction of problems that LLMs can solve.
- With repeated sampling scenarios, we consider a setup where a model can generate many candidate solutions while attempting to solve a problem. The success rate is therefore influenced:
 - Coverage: As the number of samples increases, what fraction of problems can we solve using any sample that was generated?
 - Precision: In settings where we must select a final answer from the collection of generated samples, can we identify the correct samples?



Experimental Setup

- **GSM8K**: A dataset of grade-school level *math* word problems. Select a random subset of 128 problems from the GSM8K test set.
- **MATH**: Another dataset of *math* word problems that are generally harder than those from GSM8K. Select a random subset of 128 problems.
- **MiniF2F-MATH**: A dataset of *mathematics* problems that have been formalized into *proof-checking* languages. Lean4 is used as the language and evaluates the 130 test set problems that are formalized from the MATH dataset.
- **CodeContests**: A dataset of *competitive programming* problems. Each problem has a text description, along with a set of input-output test cases (hidden from the model) that can be used to verify the correctness of a candidate solution. Enforce that models write their solutions using Python3.
- **SWE-bench Lite**: A dataset of real-world Github issues, where each problem consists of a description and a snapshot of a code repository. To solve a problem, models must edit files in the codebase (in the Lite subset of SWE-bench that we use, only a single file must be changed). Candidate solutions can be automatically checked using the repository's suite of unit tests.

Coverage Results



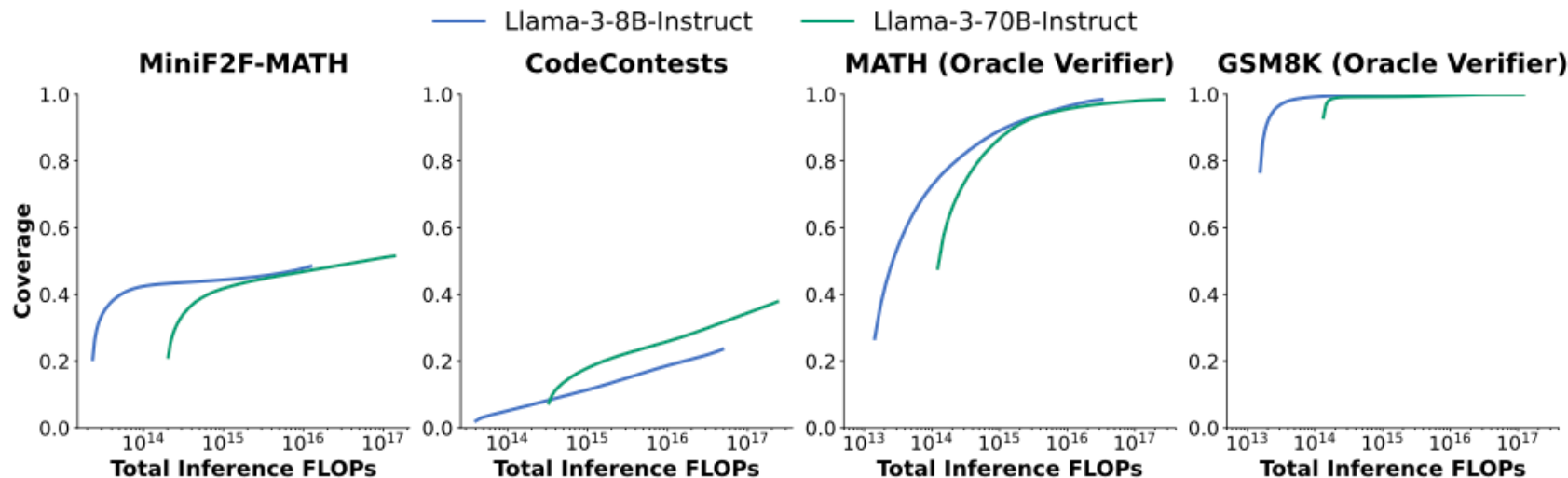
- Across five tasks, coverage (the fraction of problems solved by at least one generated sample) increases as increasing the number of samples.
- Scaling inference time compute via repeated sampling leads to consistent coverage gains across a variety of model sizes (70M-70B), families (Llama, Gemma and Pythia) and levels of post-training (Base and Instruct models).

Balance Performance and Cost

- Applied estimation of inference FLOPs:

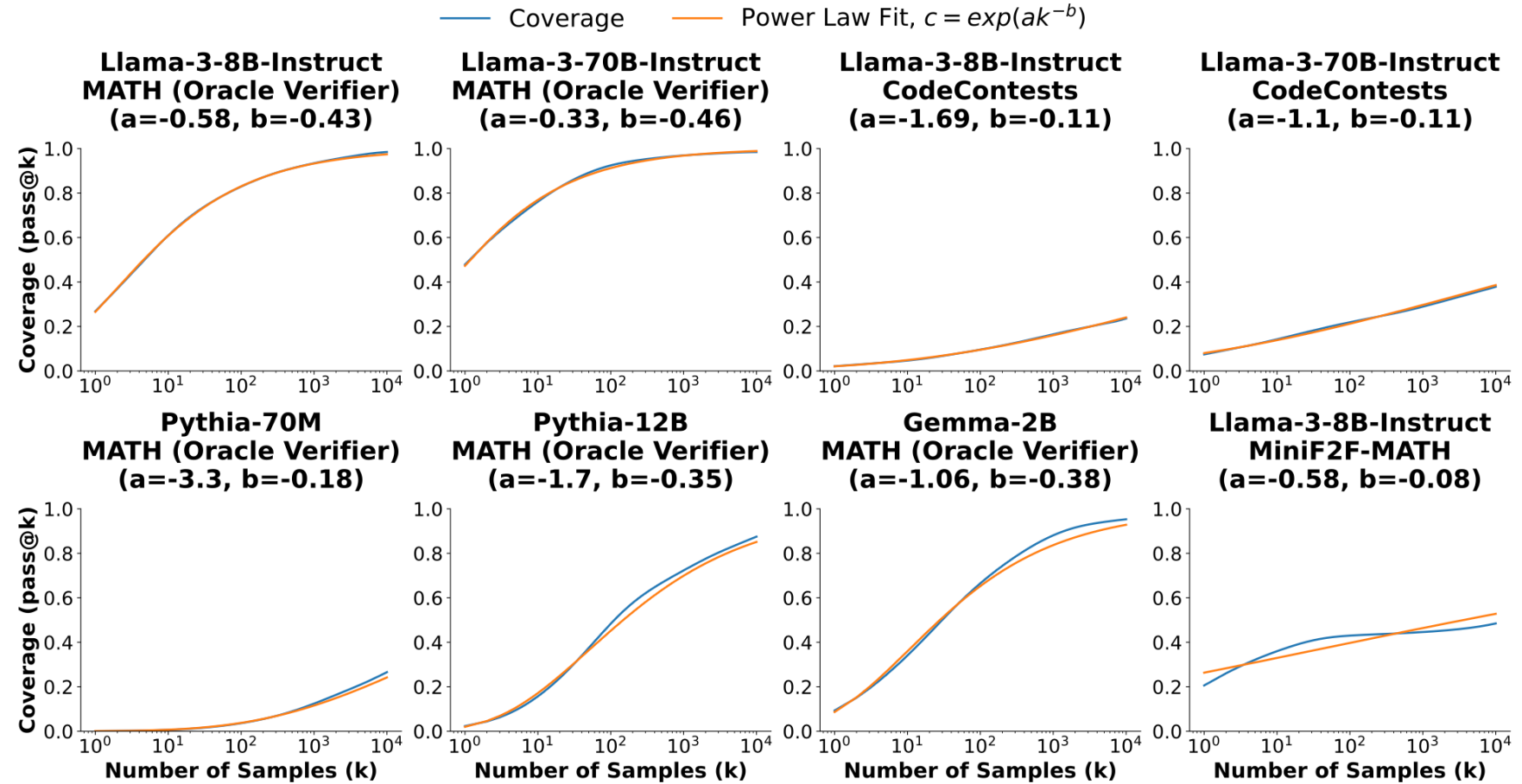
$$\begin{aligned} \text{FLOPs per token} &\approx 2 * (\text{num parameters} + 2 * \text{num layers} * \text{token dim} * \text{context length}) \\ \text{total inference FLOPs} &\approx \text{num prompt tokens} * \text{FLOPs per token} \\ &\quad + \text{num decoded tokens} * \text{FLOPs per token} * \text{num completions} \end{aligned}$$

- Comparing cost, measured in a number of inference FLOPs, and coverage for Llama-3-8B-Instruct and Llama-3-70B-Instruct: the ideal model size depends on the task, compute budget and coverage requirements.



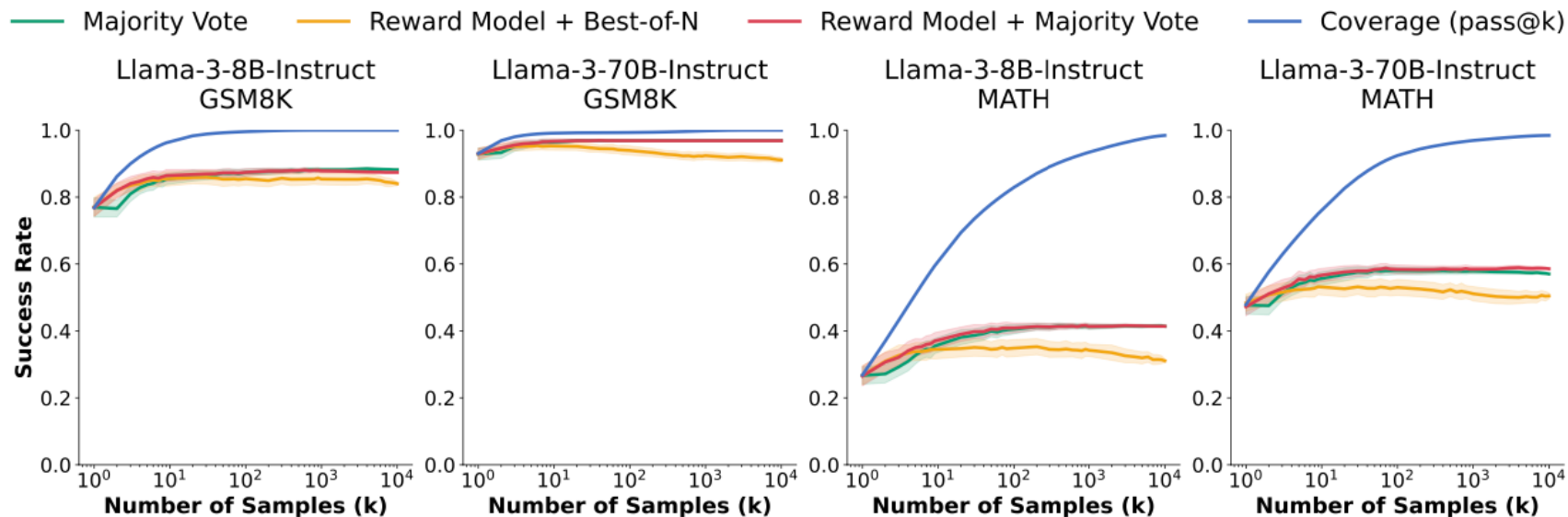
Scaling Laws for Repeated Sampling

- The log of coverage c as a function of the number of samples k :
 - $\log(c) = ak^{-b}$
 - $c = \exp(ak^{-b})$
- where $a, b \in \mathbb{R}$ are fitted model parameters.



Repeated Sampling for Precision

- Given a collection of model samples, can we identify the correct ones? For Math problems:
 - Majority vote: the most common final answer (same as self-consistency);
 - Reward Model + Best-of-N: use a **reward model** to score each solution and pick the answer from the highest-scoring sample.
 - Reward Model + Majority Vote: We calculate a majority vote where each sample is weighted by its **reward model** score.
- While success rate initially increases with the number of samples for all three methods, it plateaus around 100 samples.



More Advanced Prompt Engineering Techniques

What do we learn?

- The inference time scaling law indicates that *we should not limit the LLM to generate only one solution per problem!*
- Exploring multiple branches allows the LLM to recover from mistakes in a single generation:
 - Generate multiple candidate solutions per problem;
 - Generate multiple potential next reasoning steps given the current (partial) thought;
- Challenge: how can we select the best response from multiple candidates?
 - In most cases, we do not have an oracle scorer at inference time.

Chain-of-Thought

- *Chain-of-thought (CoT)* prompting enables complex reasoning capabilities through intermediate reasoning steps.
 - Instead of producing an answer in one step, the model is prompted to “**think step by step**”, breaking complex problems into smaller sub-problems that it solves in sequence.
- This approach can dramatically improve performance on tasks that require multi-step reasoning, such as math word problems, logical reasoning puzzles, and other tasks requiring planning or inference.
- In a standard prompt, we might ask a question and expect the model to directly output the answer. With CoT prompting, we ask the model not only for the answer but also to *show it works*.
- Transparency: the model’s reasoning is explicitly shown, which makes it easier for humans to understand or debug the model’s answer

Chain-of-Thought Prompting Examples

Question: “Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?”

If we prompt a model in a chain-of-thought manner, we might say: “**Let’s think step by step.**” This cues the model to produce reasoning:

Model’s Chain-of-Thought:

1. “Roger started with 5 balls.”
2. “He buys 2 cans of 3 tennis balls each, that is $2 \times 3 = 6$ additional balls.”
3. “Total tennis balls = $5 + 6 = 11$.”
4. “So, the answer is 11.”

Final Answer: 11

Question: “A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?”

Using CoT prompting (e.g., prefixing the answer with “**Let’s think step by step.**”), the model might reason as follows:

Model’s Chain-of-Thought:

1. “Total balls = 16.”
2. “Half of the balls are golf balls, so number of golf balls = $16 / 2 = 8$.”
3. “Half of the golf balls are blue, so number of blue golf balls = $8 / 2 = 4$.”
4. “The answer is 4.”

Final Answer: 4

Task: “Write a Python function to compute the factorial of a given integer n .”

Instead of jumping straight to the code, we prompt the model to think it through: “**Let’s break this down step by step.**”

Model’s Chain-of-Thought:

1. “Factorial of n , written as $n!$, is the product of all positive integers up to n .”
2. “We should consider a base case: $\text{factorial}(0) = 1$, and $\text{factorial}(1) = 1$.”
3. “For $n > 1$, we can compute iteratively or recursively: e.g., $\text{result} = 1$; for i in $1..n$: $\text{result} = i$.”
4. “Now I will write the code following this logic.”

Final Answer (Code):

```
def factorial(n: int) -> int:
    if n <= 1:
        return 1
    result = 1
    for i in range(1, n+1):
        result *= i
    return result
```

Few-shot CoT vs Zero-shot CoT

- *Chain-of-thought (CoT)* prompting enables complex reasoning capabilities through intermediate reasoning steps.
 - It can also be combined with both few-shot prompting and zero-shot prompting.
- **Few-Shot CoT**: Provide one or more examples (Q&A pairs) in the prompt where each answer is explained with a CoT. The model is then given a new question and is expected to mimic the format, producing a reasoning chain followed by the answer.
- **Zero-Shot CoT**: No example Q&A pairs are given. Instead, we append an instruction to the query that elicits a CoT. Simply adding a phrase like “Let’s think step by step” or “Let’s think this through logically” before the answer significantly improves results on reasoning tasks.

(a) Few-shot	(b) Few-shot-CoT
Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now? A: The answer is 11. Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there? A: (Output) The answer is 8. X	Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now? A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11. Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there? A: (Output) The juggler can juggle 16 balls. Half of the balls are golf balls. So there are $16 / 2 = 8$ golf balls. Half of the golf balls are blue. So there are $8 / 2 = 4$ blue golf balls. The answer is 4. ✓
(c) Zero-shot	(d) Zero-shot-CoT (Ours)
Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there? A: The answer (arabic numerals) is (Output) 8 X	Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there? A: Let's think step by step. (Output) There are 16 balls in total. Half of the balls are golf balls. That means that there are 8 golf balls. Half of the golf balls are blue. That means that there are 4 blue golf balls. ✓

Why Does CoT Improve Reasoning?

- **Decomposing complex tasks**: CoT forces the model to break a problem into intermediate steps, mirroring how a human might approach a multi-step question.
- **Emergent reasoning capabilities in LLM**: The ability to perform multi-step reasoning is an emergent property that only comes when the model has enough capacity and training data to have implicitly learned logical patterns. CoT prompting is essentially a way to elicit those latent reasoning skills.
- **Avoiding shallow heuristics**: Without CoT, a model might rely on superficial patterns or recall to answer a question. CoT encourages a more algorithmic approach: the model essentially simulates an algorithm (a sequence of operations) in natural language.
- **Reduction of error propagation**: By externalizing the chain of thought, there's an opportunity for the model (and the user) to catch and correct errors along the way.

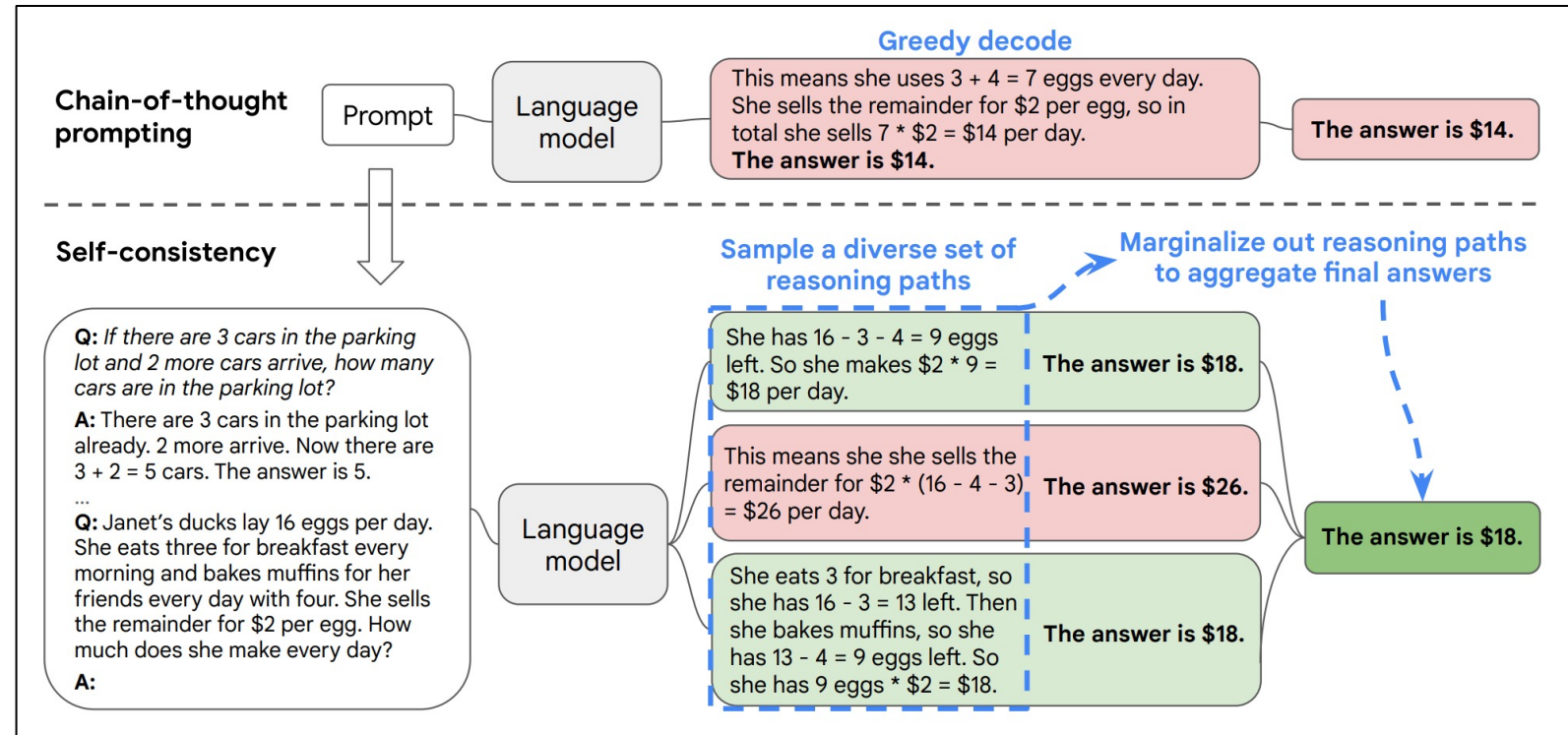
Self-Consistency

- One limitation of basic CoT prompting is that if it generates one step incorrectly, the model might *get stuck in a single flawed reasoning path*.
 - Especially when we use greedy decoding, where it always picks the most likely next token each time.
- Instead of generating one CoT, we generate many different chains-of-thought by sampling stochastically (allowing the model to explore different ways to solve the problem).
- **Intuition**: if a problem truly has a correct answer, multiple independent reasoning attempts are likely to converge on that correct answer, whereas wrong answers might be more randomly distributed.
- Comprehensively, self-consistency improves CoT by injecting diversity into the reasoning and then using a simple voting scheme to decide on the best answer.

Self-Consistency

Three steps:

1. Prompt a language model using chain-of-thought (CoT) prompting.
2. Replace the “greedy decode” in CoT prompting by sampling from the language model’s decoder to generate a diverse set of reasoning paths.
3. Marginalize the reasoning paths and aggregate by choosing the most consistent answer in the final answer set.



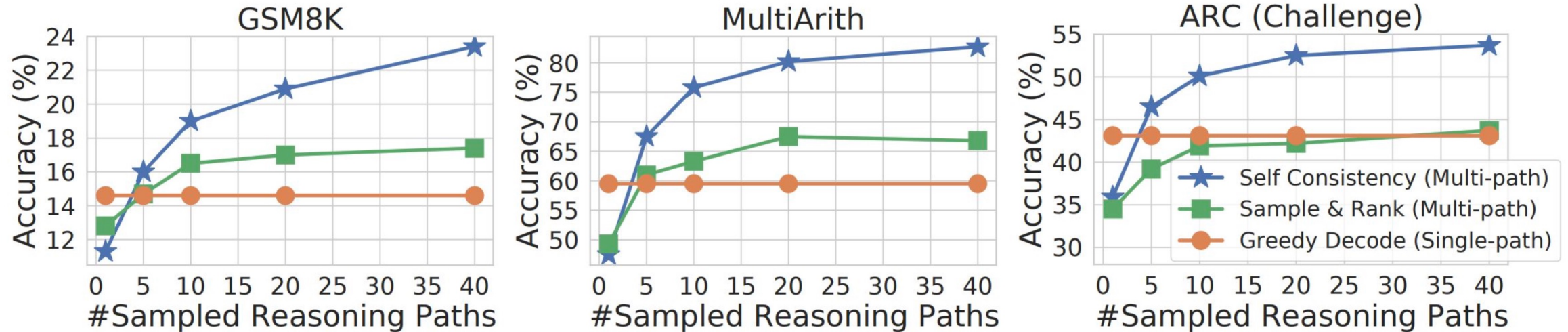
Additional notes: the selection is only based on the final answer, the reasoning paths do not need to be the same across different sampled responses.

Self-Consistency Performance Boost

	Method	AddSub	MultiArith	ASDiv	AQuA	SVAMP	GSM8K
	Previous SoTA	94.9^a	60.5 ^a	75.3 ^b	37.9 ^c	57.4 ^d	35 ^e / 55 ^g
UL2-20B	CoT-prompting	18.2	10.7	16.9	23.6	12.6	4.1
	Self-consistency	24.8 (+6.6)	15.0 (+4.3)	21.5 (+4.6)	26.9 (+3.3)	19.4 (+6.8)	7.3 (+3.2)
LaMDA-137B	CoT-prompting	52.9	51.8	49.0	17.7	38.9	17.1
	Self-consistency	63.5 (+10.6)	75.7 (+23.9)	58.2 (+9.2)	26.8 (+9.1)	53.3 (+14.4)	27.7 (+10.6)
PaLM-540B	CoT-prompting	91.9	94.7	74.0	35.8	79.0	56.5
	Self-consistency	93.7 (+1.8)	99.3 (+4.6)	81.9 (+7.9)	48.3 (+12.5)	86.6 (+7.6)	74.4 (+17.9)
GPT-3 Code-davinci-001	CoT-prompting	57.2	59.5	52.7	18.9	39.8	14.6
	Self-consistency	67.8 (+10.6)	82.7 (+23.2)	61.9 (+9.2)	25.6 (+6.7)	54.5 (+14.7)	23.4 (+8.8)
GPT-3 Code-davinci-002	CoT-prompting	89.4	96.2	80.1	39.8	75.8	60.1
	Self-consistency	91.6 (+2.2)	100.0 (+3.8)	87.8 (+7.6)	52.0 (+12.2)	86.8 (+11.0)	78.0 (+17.9)

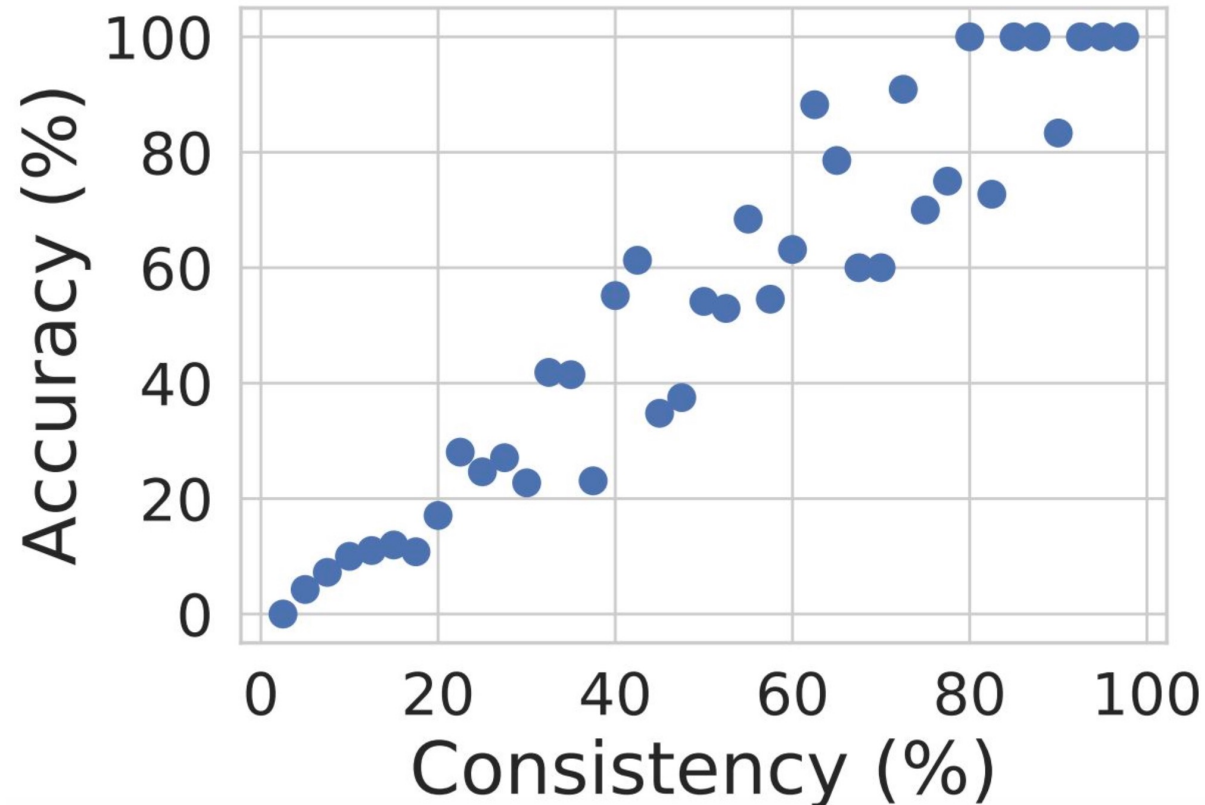
Self-consistency boosts performance across models and benchmarks.

Self-Consistency Performance Boost



- Sample-and-Rank baseline: select the response with the highest log probability;
- Self-consistency performance scales with more samples.

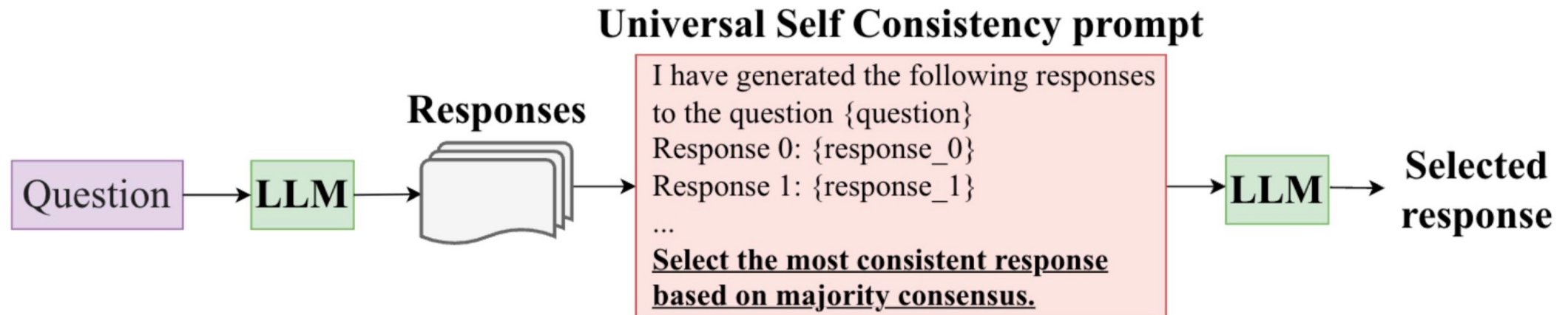
Consistency is highly correlated with the accuracy



- If more sampled responses lead to the same final answer:
 - The LLM is more certain with its predicted conclusion.
 - The aggregated solution is more likely to be correct.

Universal Self-Consistency

- Limitation of self-consistency decoding: require an answer extraction process;
- Universal self-consistency: ask the LLM to perform consistency-based selection.



<https://arxiv.org/abs/2311.17311>

Universal Self-Consistency Performance Boost

Dataset	Approach	ROUGE-1	ROUGE-2	ROUGE-Lsum	BERTScore
GovReport	Greedy decoding	38.8	16.9	33.8	62.7
	Random	38.5	16.9	33.6	62.6
	USC	40.2	17.4	35.1	62.8
SummScreen	Greedy decoding	30.6	7.5	19.1	58.7
	Random	30.2	7.3	19.0	58.6
	USC	31.7	7.8	19.8	58.3

- Results on long-context summarization benchmarks with **PaLM 2-L**. Since the outputs are in free-form, the standard self-consistency is not applicable. Universal Self-Consistency (USC) consistently improves over the baselines on summary quality.

Universal Self-Consistency Performance Boost

Model	Approach	GPT-judge	GPT-info
PaLM 2-L	Greedy decoding	62.1	95.1
	Random	62.9	94.6
	USC	67.7	99.0
gpt-3.5-turbo	Greedy decoding	79.8	99.7
	Random	80.6	99.3
	USC	82.5	99.6

- Accuracy on the TruthfulQA benchmark. Since the answer is in free-form, the standard self-consistency is not applicable. USC overall has the highest truthfulness and informativeness over the baselines.

USC vs SC

Model	Approach	GSM8K	MATH
PaLM 2-L	Greedy decoding	85.7	30.8
	Random	82.9	28.0
	SC (Wang et al., 2022)	90.4	37.9
	USC	90.2	37.4
gpt-3.5-turbo	Greedy decoding	73.4	33.2
	Random	68.5	26.3
	SC	78.5	38.0
	USC	77.8	38.1

- Accuracy on mathematical reasoning benchmarks. USC and SC consistently improve over the greedy decoding and random selection. USC performance is generally comparable to SC.

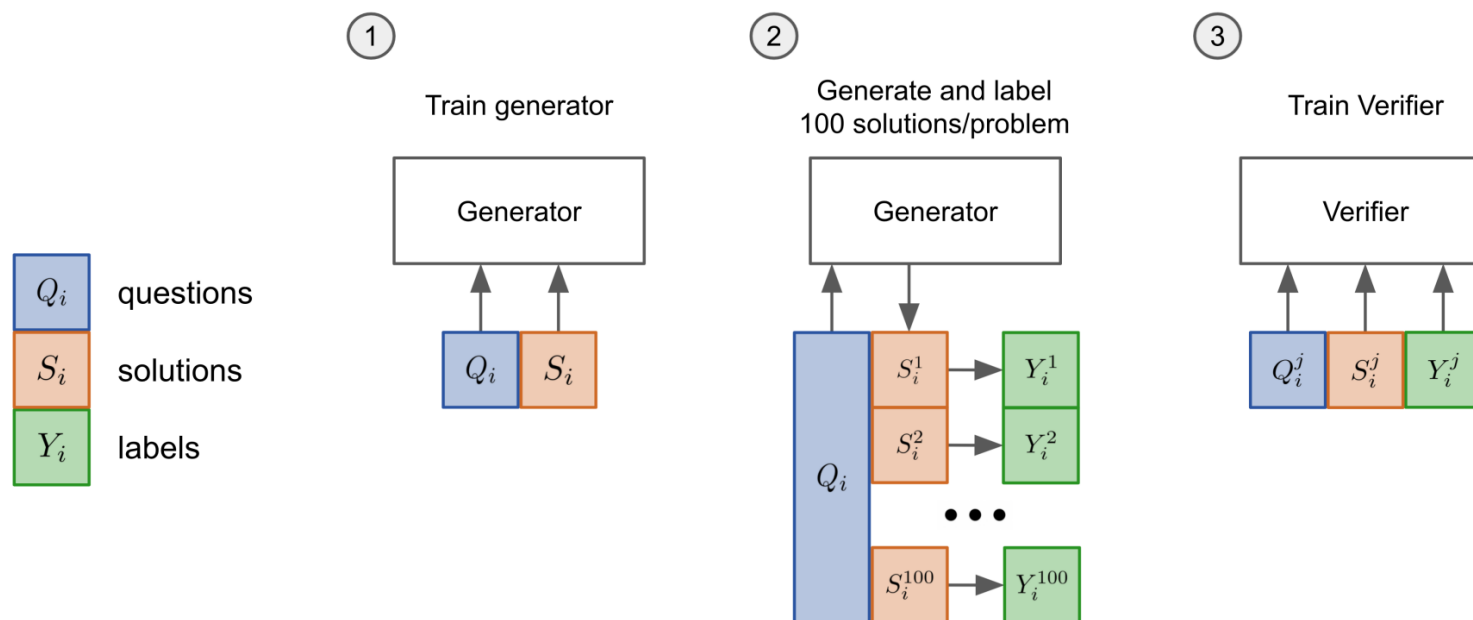
USC Summary

- Universal Self-Consistency (USC) improves the performance on open-ended generation (summarization, QA), where the original self-consistency is inapplicable.
- USC matches self-consistency performance on math reasoning and coding.
 - USC does not require answer extraction and code execution.
- USC performance is bounded by the long-context capability.

Train LLM Rankers

- General idea: train verifiers/reward models to judge the correctness of model-generated solutions and search against these verifiers at test time.
- Conditioned on the problem and a candidate solution, the verifier outputs the probability that the solution is correct.
- Training solutions are labeled as correct or incorrect based solely on whether they reach the correct final answer.

Train LLM Rankers



1. Finetune a model (the “generator”) for a few epochs on the training set.
2. Sample some completions from the generator for each training problem and label each solution as correct or incorrect.
3. Train a verifier for a single epoch on this dataset.

Train LLM Rankers

- We can train two types of LLM-based verifiers/reward models:
 - **Outcome-supervised Reward Model (ORM)**: verify at the solution level.
 - **Process-supervised Reward Model (PRM)**: verify at the step level for each solution.

The denominator of a fraction is 7 less than 3 times the numerator. If the fraction is equivalent to $\frac{2}{5}$, what is the numerator of the fraction? (Answer:)

   Let's call the numerator x .

   So the denominator is $3x-7$.

   We know that $x/(3x-7) = 2/5$.

   So $5x = 2(3x-7)$.

   $5x = 6x - 14$.

   So $x = 7$.

Collect feedback for each step in a solution.

Train ORM

- Uniformly sample a fixed number of solutions per problem from the generator.
- Train the ORM to predict whether each solution is correct or incorrect.
- In practice, one usually can determine correctness by automatically checking the final answer, but in principle these labels could be provided by humans.
- At test time, we can use the ORM's prediction at the final token as the overall score for the solution.
- Noted the automatic grading used to determine ORM targets is not perfectly reliable: false positives solutions that reach the correct answer with incorrect reasoning will be misgraded.

Train PRM

- Train PRMs to predict the correctness of each step after the last token in each step.
- This prediction takes the form of a single token, and we maximize the log-likelihood of these target tokens during training.
- The PRM can, therefore, be trained in a standard language model pipeline.
- To determine the step-level predictions at test time, it suffices to perform a single PRM forward pass over the whole solution.
- Define the PRM score for a solution to be the probability that every step is correct under the PRM.
- Implement this as the product of the correctness probabilities for each step.

PRM at Inference Time

Let

$$x^8 + 3x^4 - 4 = p_1(x)p_2(x) \cdots p_k(x),$$

where each non-constant polynomial $p_i(x)$ is monic with integer coefficients, and cannot be factored further over the integers. Compute $p_1(1) + p_2(1) + \cdots + p_k(1)$.

I notice that the given polynomial has even degree and only even powers of x , so I can try to make a substitution to simplify it.

Let $y = x^4$, then the polynomial becomes $y^2 + 3y - 4$, which is a quadratic equation.

I can factor this quadratic equation as $(y + 4)(y - 1)$, so the original polynomial is $(x^4 + 4)(x^4 - 1)$.

Now I need to factor each of these factors further, if possible, over the integers.

For the first factor, $x^4 + 4$, I recall the Sophie Germain identity, which says that $a^4 + 4b^4 = (a^2 + 2b^2 + 2ab)(a^2 + 2b^2 - 2ab)$.

Applying this identity with $a = x$ and $b = 1$, I get $x^4 + 4 = (x^2 + 2x + 2)(x^2 - 2x + 2)$.

For the second factor, $x^4 - 1$, I can use the difference of squares formula, which says that $a^2 - b^2 = (a + b)(a - b)$.

Applying this formula with $a = x^2$ and $b = 1$, I get $x^4 - 1 = (x^2 + 1)(x^2 - 1)$.

Then I can apply the difference of squares formula again with $a = x$ and $b = 1$, to get $x^2 - 1 = (x + 1)(x - 1)$.

So the original polynomial is $(x^4 + 4)(x^4 - 1) = (x^2 + 2x + 2)(x^2 - 2x + 2)(x^2 + 1)(x + 1)(x - 1)$.

These are all the irreducible factors over the integers, so $p_1(x) = x^2 + 2x + 2$, $p_2(x) = x^2 - 2x + 2$, $p_3(x) = x^2 + 1$, $p_4(x) = x + 1$, and $p_5(x) = x - 1$.

To find the sum of their values at $x = 1$, I just need to plug in $x = 1$ into each factor and add them up.

I get $p_1(1) = 1^2 + 2 \cdot 1 + 2 = 5$, $p_2(1) = 1^2 - 2 \cdot 1 + 2 = 1$, $p_3(1) = 1^2 + 1 = 2$, $p_4(1) = 1 + 1 = 2$, and $p_5(1) = 1 - 1 = 0$.

So the final answer is $5 + 1 + 2 + 2 + 0 = 10$.

Answer: 10

Let

$$x^8 + 3x^4 - 4 = p_1(x)p_2(x) \cdots p_k(x),$$

where each non-constant polynomial $p_i(x)$ is monic with integer coefficients, and cannot be factored further over the integers. Compute $p_1(1) + p_2(1) + \cdots + p_k(1)$.

This problem is about factoring a polynomial over the integers, which can be a tricky task.

One possible strategy is to look for rational roots of the polynomial, using the rational root theorem.

However, that would involve checking a lot of possible factors of -4 , and there is no guarantee that the polynomial has any rational roots at all.

Another possible strategy is to use some algebraic tricks to simplify the polynomial or reduce its degree.

I notice that the polynomial has only even powers of x , so I can try to substitute $y = x^2$ and see what happens.

Then the polynomial becomes $y^4 + 3y^2 - 4$, which is a quadratic equation in y^2 .

I can try to factor this quadratic equation by looking for two numbers that multiply to -4 and add to 3 .

Those numbers are 4 and -1 , so I can write $y^4 + 3y^2 - 4 = (y^2 + 4)(y^2 - 1)$.

Now I can substitute back $x^2 = y$ and get $x^8 + 3x^4 - 4 = (x^4 + 4)(x^4 - 1)$.

This looks promising, because I have reduced the degree of the factors from 8 to 4 .

I can try to factor each of these factors further, using the difference of squares formula.

I get $x^8 + 3x^4 - 4 = (x^2 + 2)(x^2 - 2)(x^2 + 1)(x^2 - 1)$.

I can apply the difference of squares formula again to the last factor and get $x^8 + 3x^4 - 4 = (x^2 + 2)(x^2 - 2)(x^2 + 1)(x + 1)(x - 1)$.

Now I have factored the polynomial completely into monic linear and quadratic factors with integer coefficients.

These are the $p_i(x)$'s that the problem is asking for.

To find the sum of their values at $x = 1$, I just need to plug in $x = 1$ into each factor and add them up.

I get $p_1(1) + p_2(1) + \cdots + p_k(1) = (1^2 + 2)(1^2 - 2)(1^2 + 1)(1 + 1)(1 - 1)$.

Simplifying, I get $p_1(1) + p_2(1) + \cdots + p_k(1) = (3)(-1)(2)(2)(0)$.

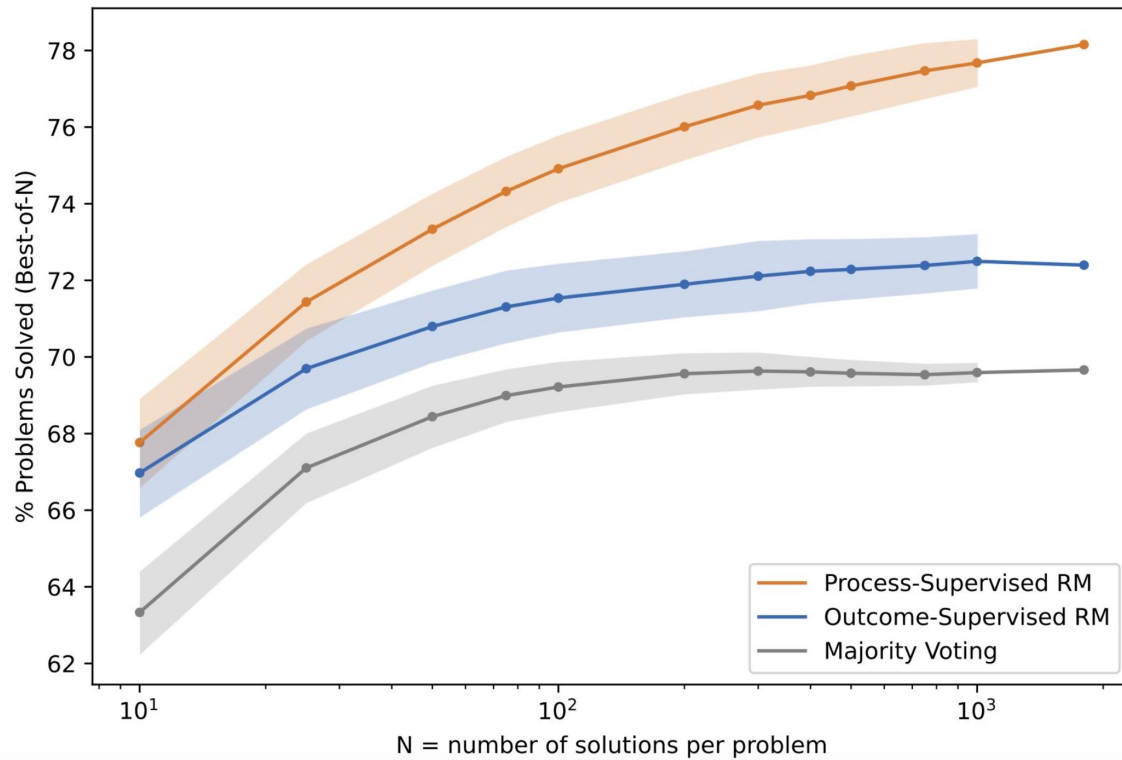
Multiplying, I get $p_1(1) + p_2(1) + \cdots + p_k(1) = 0$.

Answer: 0

- Two solutions to the same problem, graded by the PRM.
- The solution on the left is correct while the solution on the right is incorrect.
- A green background indicates a high PRM score, and a red background indicates a low score.
- The PRM correctly identifies the mistake in the incorrect solution.

Strong LLM-based Verifiers

	ORM	PRM	Majority Voting
% Solved (Best-of-1860)	72.4	78.2	69.6



- (Strong) LLM-based verifiers outperform consistency-based selection.
- Process-supervised reward model (PRM) scales better with more samples.
- The performance is highly dependent on the verifier quality.
- The same verifier might not generalize across tasks.

Tree of Thoughts

- Tree-of-Thought (ToT) prompting is an extension of CoT that allows the model to *explore multiple reasoning paths in parallel and hierarchically*, rather than one linear chain.
- ToT lets the model branch out like a tree, consider different possible steps at each juncture, evaluate them, and choose the most promising path.
- Instead of the model producing one next step, it can propose several candidate next steps (thoughts) at a certain point.
- Then, a search algorithm (like breadth-first search or depth-first search) guided by the model's own evaluation of partial solutions will explore these branches.
- To implement ToT prompting, we can use some structured prompting where the model is asked to propose possible next steps and also to critique or score partial solutions, e.g.,
“Propose three possible next steps you could take.” Then: “Evaluate each: label it as (sure) likely to lead to solution, (maybe), or (impossible).” Then: “Choose the best next step and continue.”

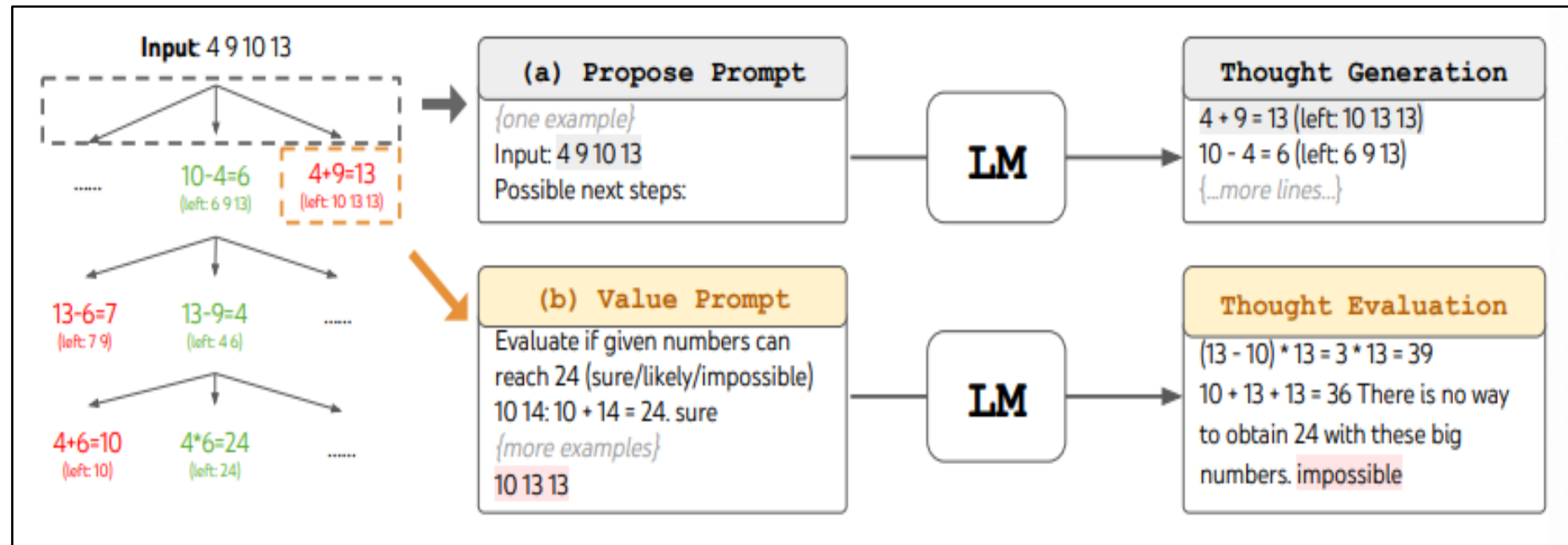
Tree of Thoughts

- To use ToT prompting in practice, we can conceptually do:
 - **Thought proposals**: Ask the model to generate multiple possible next steps/thoughts.
 - **Evaluation**: Ask the model (or use a heuristic) to judge which proposed thought seems most promising toward solving the problem.
 - **Expansion**: Take the best thought, append it to the chain, and repeat from step 1.
 - **Backtracking**: If you reach a dead-end (the model says “impossible” or no solution), go back up the tree and try a different branch.
- For different problems, the choice depends on task structure:
 - **Breadth-First Search** keeps the top-k candidates at each depth level and is suited for shallow problems.
 - **Depth-First Search** explores one path deeply before backtracking and is better for deeper search spaces.

Tree of Thoughts (ToT)

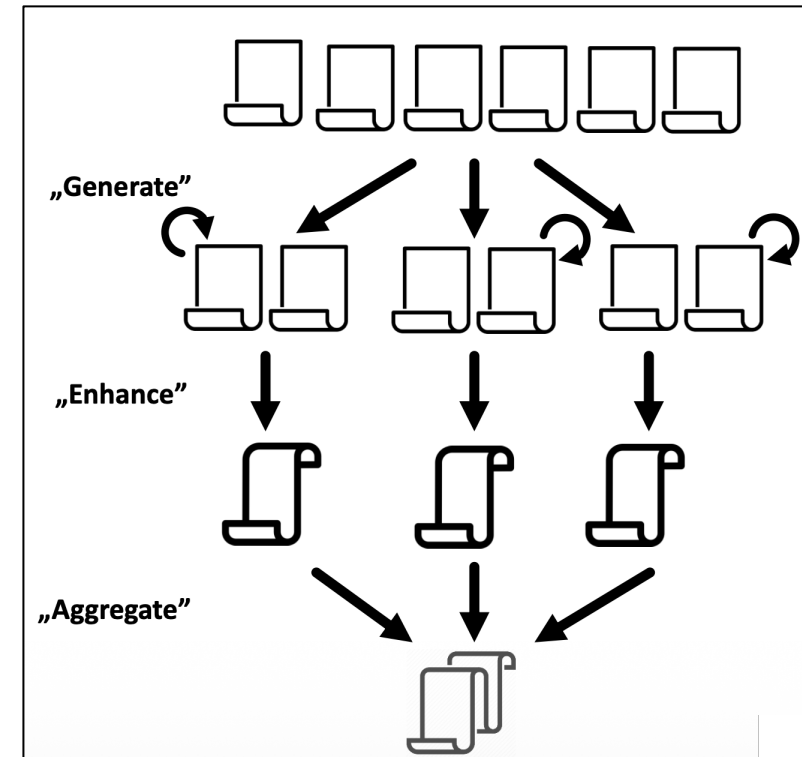
- Example: Game of 24:

- To perform BFS in ToT for the Game of 24 task, the LLM is prompted to evaluate each thought candidate as sure/maybe/impossible w.r.t reaching 24.
- The aim is to promote correct partial solutions that can be verified within a few lookahead trials, eliminate impossible partial solutions based on too big/small commonsense, and keep the rest maybe.
- Values are sampled 3 times for each thought.



Graph of Thoughts

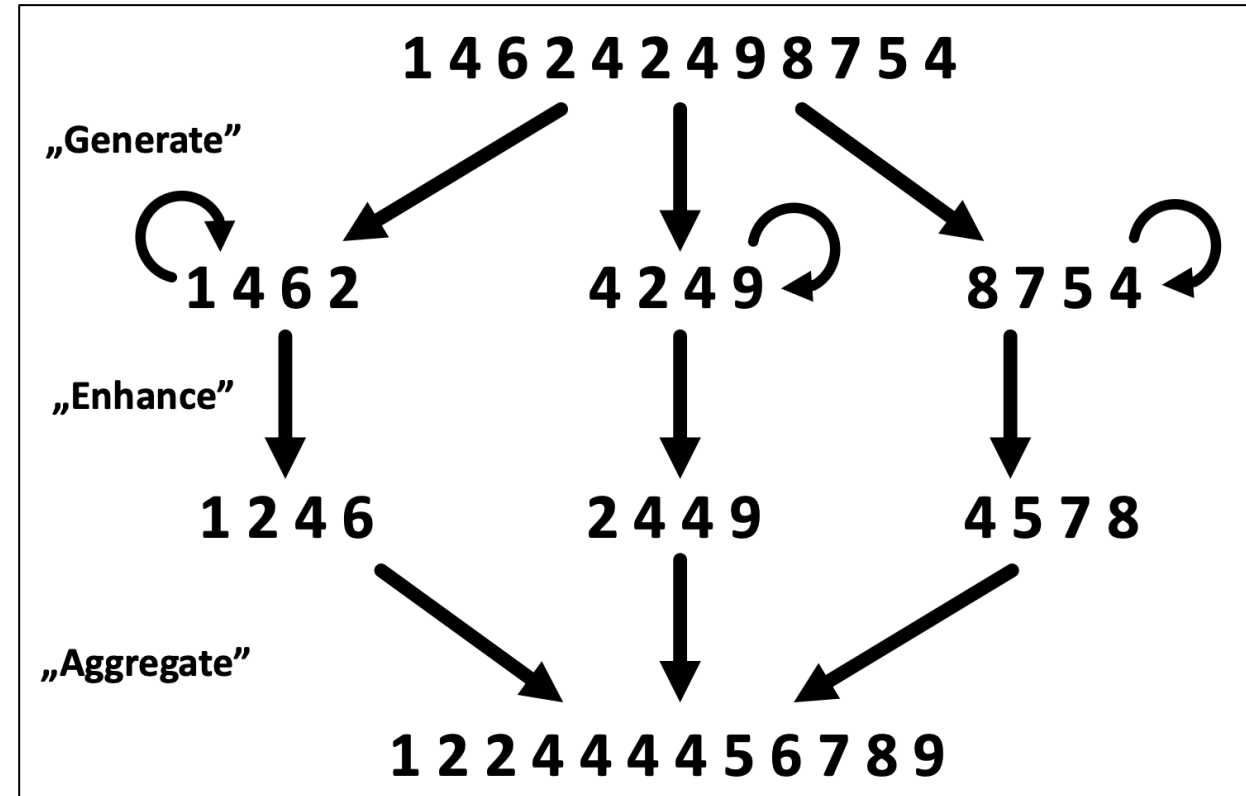
- ToT constrains reasoning to a tree, i.e., each thought has at most one parent. GoT removes this restriction, modeling reasoning as an **arbitrary directed graph** $G = (V, E)$ where thoughts are vertices and dependencies are edges.
- **Generation** creates new thoughts from an existing one (branching, as in CoT/ToT).
- **Aggregation** merges k thoughts into one single new thought.
- **Refinement** iteratively improves a thought via self-loops, enabling feedback cycles.



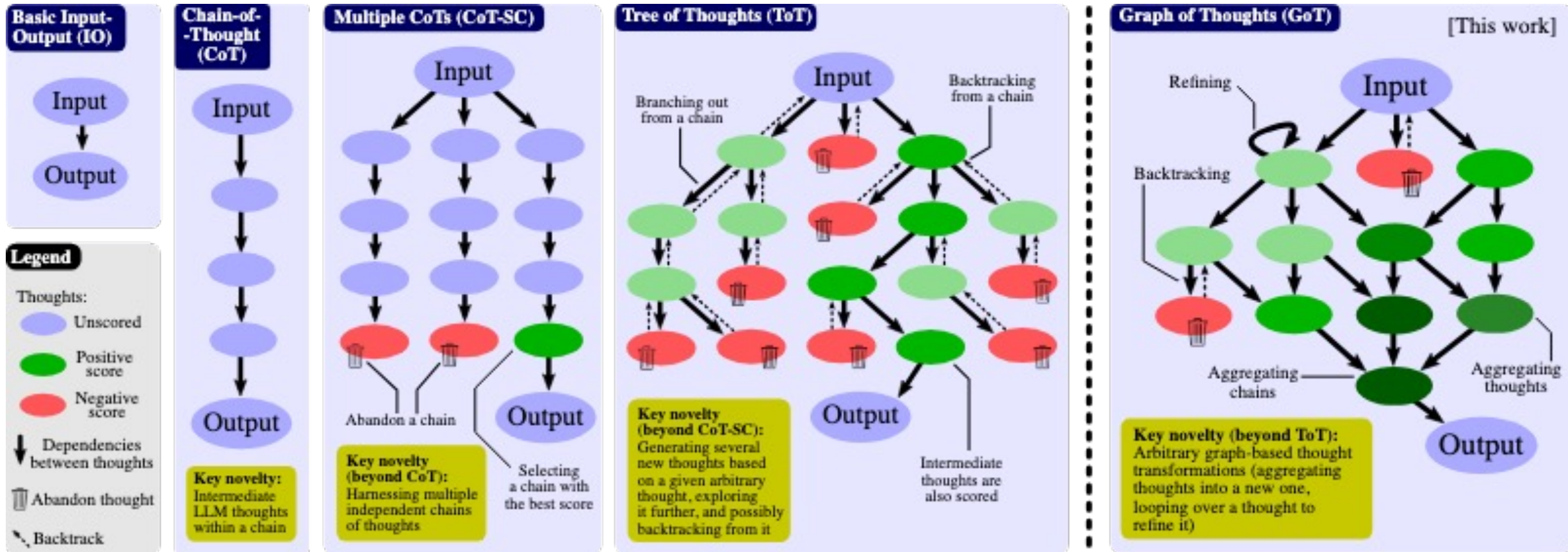
Graph of Thoughts

Example for sorting:

- **Generation** creates new thoughts from an existing one (branching, as in CoT/ToT).
In sorting example: splitting a 64-number array into four 16-element chunks for independent sorting.
- **Aggregation** merges k thoughts into one new thought. *In sorting example:* merging two independently sorted 16-element subarrays into a sorted 32-element array, analogous to the merge step in merge sort.
- **Refinement** iteratively improves a thought via self-loops, enabling feedback cycles. *In sorting example:* re-checking and correcting a sorted subarray.



Some Comparison and Summary



<https://github.com/spcl/graph-of-thoughts/blob/main/paper/pics/preview.svg>

Summary & Reference

- Inference scaling law indicates that we can scale the inference-time compute by sampling multiple branches in the solution space for performance boost.
- Consistency-based selection: a simple, effective, and general principle.
- When LLM self-evaluation works well: search in the partial solution space can help.

- <https://arxiv.org/pdf/2407.21787>
- <https://arxiv.org/abs/2311.17311>
- <https://arxiv.org/abs/2305.20050>
- <https://arxiv.org/pdf/2305.10601>
- <https://arxiv.org/abs/2308.09687>
- https://rdi.berkeley.edu/adv-llm-agents/slides/inference_time_techniques_lecture_sp25.pdf